

INTERNSHIP PROJECT REPORT

DESIGN AND IMPLEMENTATION OF A DIGITAL VENDING MACHINE USING VERILOG HDL

Submitted By:

POOJA RAJEEVAYYA KABBINAKANTIMATH

JUNE 2026

ABSTRACT

A vending machine is an automated system that dispenses products after receiving payment from users. This project presents the design and implementation of a Digital Vending Machine using Verilog Hardware Description Language (HDL) based on a Finite State Machine (FSM) architecture. The system allows users to select products, make payments through either coin insertion or online transactions, receive products upon successful payment, and obtain change when excess money is inserted. The vending machine also supports transaction cancellation with automatic refund of the inserted amount. Coin debouncing is incorporated to prevent multiple registrations of a single coin insertion. The design is verified through simulation using Xilinx Vivado and is suitable for implementation on FPGA platforms such as Artix-7.

Keywords: Verilog HDL, FPGA, Finite State Machine, Vending Machine, Coin Debouncing, Online Payment, Digital Design.

CHAPTER 1

INTRODUCTION

1.1 Overview

Vending machines have become an essential part of modern automated retail systems. They provide quick and convenient access to products without requiring human intervention. Traditional vending machines rely on mechanical systems, whereas modern machines utilize digital electronics and embedded systems for improved functionality and reliability.

This project focuses on designing a digital vending machine using Verilog HDL. The system is implemented as a Finite State Machine (FSM) that controls product selection, payment processing, dispensing, and transaction management.

1.2 Motivation

The growing demand for automation has increased the need for intelligent vending systems. Digital vending machines reduce human effort, improve transaction accuracy, and provide a user-friendly purchasing experience. Implementing such systems using FPGA technology offers flexibility, high speed, and reliability.

1.3 Objectives

The objectives of this project are:

- To design a vending machine using Verilog HDL.
- To implement product selection functionality.
- To support both coin-based and online payments.
- To calculate and return change automatically.
- To implement transaction cancellation with refund.
- To incorporate coin debouncing mechanisms.

- To verify the functionality through simulation.

CHAPTER 2

SYSTEM REQUIREMENTS

2.1 Functional Requirements

The vending machine should:

1. Allow users to select products.
2. Accept coin payments.
3. Support online payment transactions.
4. Dispense products after successful payment.
5. Return excess amount as change.
6. Refund money if the transaction is cancelled.
7. Prevent false multiple coin detections.

2.2 Product List

Product	Price (₹)
Pen	10
Notebook	50
Water Bottle	20
Lays	20
Coke	35

CHAPTER 3

SYSTEM DESIGN

3.1 Block Diagram

3.2 Finite State Machine

The vending machine is implemented using five states:

State 0 – IDLE

- Initial state after reset.
- Waits for user input.

State 1 – SELECT

- User selects a product.
- Product price is loaded.

State 2 – PAYMENT

- Accepts coins or online payment.
- Checks if sufficient payment has been received.

State 3 – DISPENSE

- Dispenses selected product.
- Calculates and returns change.

State 4 – CANCEL

- Refunds inserted amount.
- Returns to idle state.

CHAPTER 4

IMPLEMENTATION

4.1 Inputs

Signal	Description
i_clk	System Clock
i_rst	System Reset
i_start	Start Transaction
i_cancel	Cancel Transaction
i_product_code	Product Selection
i_coin_valid	Coin Detection
i_coin_value	Coin Value
i_online_payment	Online Payment Signal

4.2 Verilog Code:

```

`timescale 1ns / 1ps

module VendingMachine(
    input wire i_clk,
    input wire i_rst,
    input wire i_start,
    input wire i_cancel,
    input wire [2:0] i_product_code,
    input wire i_coin_valid,
    input wire [6:0] i_coin_value,
    input wire i_online_payment,
    output reg [2:0] o_state,
    output reg o_dispense_product,
    output reg [6:0] o_product_price,
    output reg [6:0] o_total_amount,
    output reg [6:0] o_return_change
);

//=====

// PRODUCT PRICES

//=====

parameter PEN_PRICE    = 7'd10;
parameter NOTEBOOK_PRICE = 7'd50;
parameter WATER_PRICE  = 7'd20;
parameter LAYS_PRICE   = 7'd20;
parameter COKE_PRICE   = 7'd35;

//=====

// FSM STATES

//=====

parameter IDLE_STATE    = 3'd0;
parameter SELECT_STATE  = 3'd1;
parameter PAYMENT_STATE = 3'd2;

```

```

parameter DISPENSE_STATE = 3'd3;
parameter CANCEL_STATE  = 3'd4;

reg [2:0] next_state;

reg coin_prev;

//=====

// PRODUCT PRICE

//=====

always @(*)
begin
    case(i_product_code)
        3'b000: o_product_price = PEN_PRICE;
        3'b001: o_product_price = NOTEBOOK_PRICE;
        3'b010: o_product_price = WATER_PRICE;
        3'b011: o_product_price = LAYS_PRICE;
        3'b100: o_product_price = COKE_PRICE;
        default: o_product_price = 0;
    endcase
end

//=====

// STATE REGISTER

//=====

always @(posedge i_clk or posedge i_rst)
begin
    if(i_rst)
        o_state <= IDLE_STATE;
    else
        o_state <= next_state;
    end
end

//=====

```

```
// NEXT STATE LOGIC

//=====

always @(*)
begin
    next_state = o_state;
    case(o_state)
        IDLE_STATE:
            begin
                if(i_start)
                    next_state = SELECT_STATE;
            end
        SELECT_STATE:
            begin
                next_state = PAYMENT_STATE;
            end
        PAYMENT_STATE:
            begin
                if(i_cancel)
                    next_state = CANCEL_STATE;
                else if(i_online_payment)
                    next_state = DISPENSE_STATE;
                else if(o_total_amount >= o_product_price)
                    next_state = DISPENSE_STATE;
            end
        DISPENSE_STATE:
            next_state = IDLE_STATE;
        CANCEL_STATE:
            next_state = IDLE_STATE;

        default:
```

```

        next_state = IDLE_STATE;
    endcase
end

//=====

// COIN DEBOUNCE + AMOUNT

//=====

always @(posedge i_clk or posedge i_rst)
begin
    if(i_rst)
    begin
        o_total_amount <= 0;
        coin_prev <= 0;
    end
    else
    begin
        coin_prev <= i_coin_valid;
        if(o_state == PAYMENT_STATE)
        begin
            if(i_coin_valid && !coin_prev)
                o_total_amount <= o_total_amount + i_coin_value;
        end
        else if(o_state == IDLE_STATE)
        begin
            o_total_amount <= 0;
        end
    end
end

//=====

```



```

// OUTPUT LOGIC

//=====

always @(posedge i_clk or posedge i_rst)
begin
    if(i_rst)
    begin
        o_dispense_product <= 0;
        o_return_change <= 0;
    end
    else
    begin
        case(o_state)
            DISPENSE_STATE:
            begin
                o_dispense_product <= 1;
                if(i_online_payment)
                    o_return_change <= 0;
                else
                    o_return_change <=
                        o_total_amount - o_product_price;
            end
            CANCEL_STATE:
            begi
                o_dispense_product <= 0;
                // Refund inserted money
                o_return_change <= o_total_amount;
            end
            default:
            begin
                o_dispense_product <= 0;

```

```
        end
    endcase
end
end
endmodule
```

4.3 Testbench Code

```
`timescale 1ns / 1ps
module VendingMachine_tb;
    reg i_clk;
    reg i_rst;
    reg i_start;
    reg i_cancel;
    reg [2:0] i_product_code;
    reg i_coin_valid;
    reg [6:0] i_coin_value;
    reg i_online_payment;
    wire [2:0] o_state;
    wire o_dispense_product;
    wire [6:0] o_product_price;
    wire [6:0] o_total_amount;
    wire [6:0] o_return_change;
    VendingMachine DUT(
        .i_clk(i_clk),
        .i_rst(i_rst),
        .i_start(i_start),
        .i_cancel(i_cancel),
        .i_product_code(i_product_code),

        .i_coin_valid(i_coin_valid),
        .i_coin_value(i_coin_value),
```

```

.i_online_payment(i_online_payment),
.o_state(o_state),
.o_dispense_product(o_dispense_product),
.o_product_price(o_product_price),
.o_total_amount(o_total_amount),
.o_return_change(o_return_change)
);

//=====

// CLOCK

//=====

always #5 i_clk = ~i_clk;

//=====

// COIN INSERT TASK

//=====

task insert_coin;
input [6:0] value;
begin
    @(posedge i_clk);
    i_coin_valid = 1;
    i_coin_value = value;
    @(posedge i_clk);
    i_coin_valid = 0;
    i_coin_value = 0;
end

endtask;

//=====

// TEST CASES

//=====

initial

```

```
begin
    i_clk = 0;
    i_rst = 1;
    i_start = 0;
    i_cancel = 0;
    i_product_code = 0;
    i_coin_valid = 0;
    i_coin_value = 0;
    i_online_payment = 0;
    #20;
    i_rst = 0;
    //=====
    // PEN (₹10)
    //=====
    i_product_code = 3'b000;
    i_start = 1;
    #10 i_start = 0;
    insert_coin(10);
    #40;
    //=====
    // NOTEBOOK (₹50)
    //=====
    i_product_code = 3'b001;
    i_start = 1;
    #10 i_start = 0;
    insert_coin(50);
    #40;
    //=====
    // COKE (₹35)
```

```
// Insert ₹50 -> Change ₹15

//=====

i_product_code = 3'b100;
i_start = 1;
#10 i_start = 0;
insert_coin(50);
#40;

//=====

// ONLINE PAYMENT

//=====

i_product_code = 3'b011;
i_start = 1;
#10 i_start = 0;
#10;
i_online_payment = 1;
#20;
i_online_payment = 0;
#40;

//=====

// CANCEL TEST

// Notebook ₹50

// Insert ₹20

// Refund ₹20

//=====

i_product_code = 3'b001;
i_start = 1;
#10 i_start = 0;

insert_coin(20);
#10;
```

```
i_cancel = 1;
#10;
i_cancel = 0
#100;
$finish;
end

//=====

// MONITOR

//=====

initial
begin
$monitor("TIME=%0t  STATE=%0d  PRICE=%0d  AMOUNT=%0d  DISPENSE=%0b
CHANGE=%0d"
    $time,
    o_state,
    o_product_price,
    o_total_amount,
    o_dispense_product,
    o_return_change);
end
endmodule
```

4.4 Outputs

Signal	Description
o_state	Current FSM State
o_dispense_product	Product Dispense Signal
o_product_price	Selected Product Price
o_total_amount	Total Inserted Amount
o_return_change	Returned Change

CHAPTER 5

COIN DEBOUNCING

Mechanical switches may generate multiple pulses when pressed once. Coin debouncing prevents such unwanted multiple detections.

The design uses edge detection:

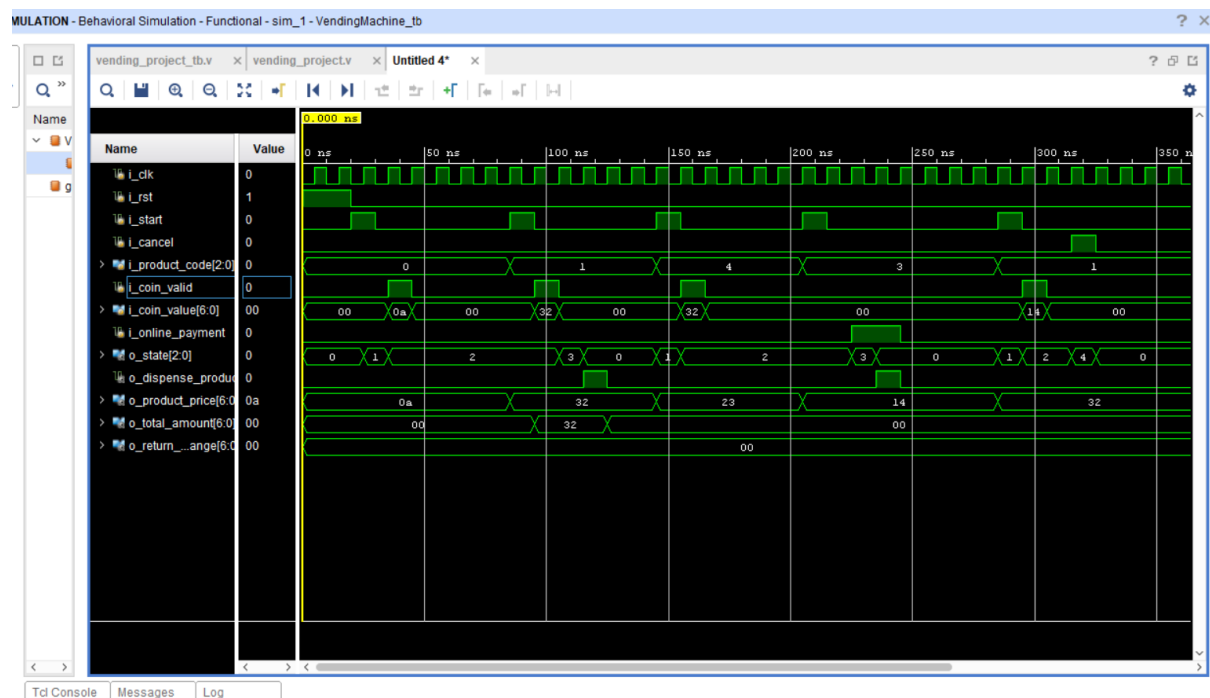
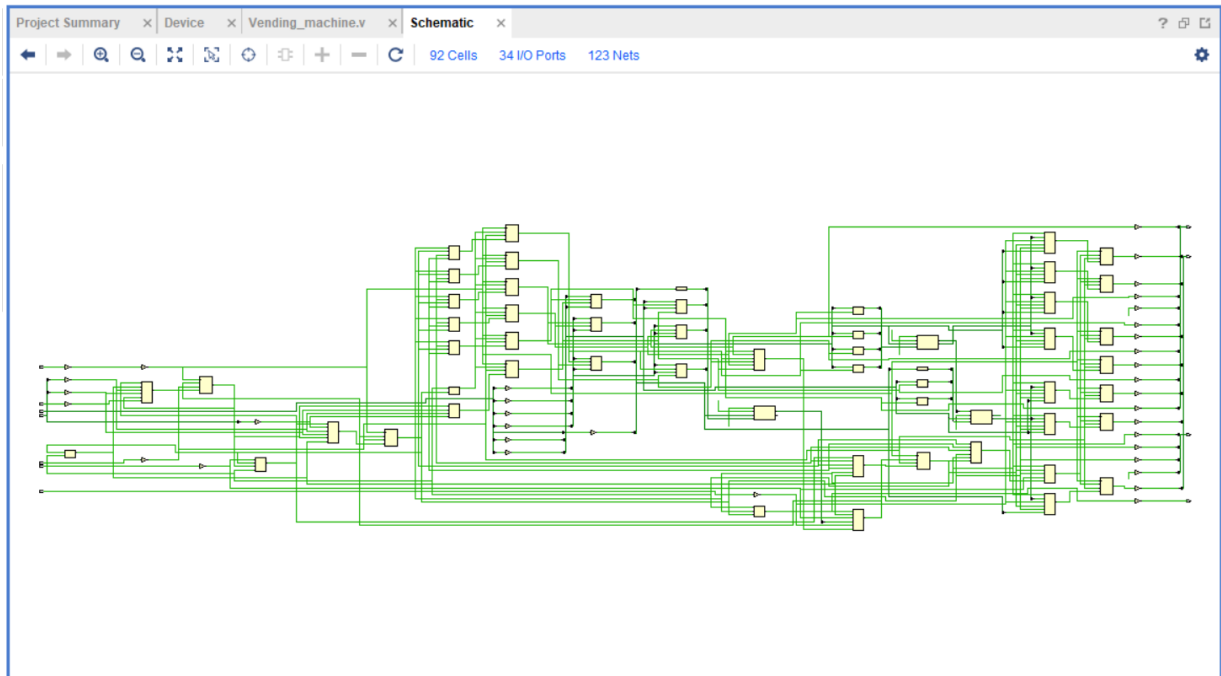
- Previous coin status is stored.
- Coin value is added only on the rising edge.
- Ensures one coin insertion is counted exactly once.

Advantages:

- Improved reliability.
- Accurate transaction processing.
- Elimination of duplicate coin counts.

CHAPTER 6

SIMULATION RESULTS



Test Case 1: Pen Purchase

Product Price = ₹10

Inserted Amount = ₹10

Result:

- Product Dispensed
- Change Returned = ₹0

Test Case 2: Notebook Purchase

Product Price = ₹50

Inserted Amount = ₹50

Result:

- Product Dispensed
- Change Returned = ₹0

Test Case 3: Coke Purchase

Product Price = ₹35

Inserted Amount = ₹50

Result:

- Product Dispensed
- Change Returned = ₹15

Test Case 4: Online Payment

Product = Lays

Payment Mode = Online

Result:

- Product Dispensed
- Change Returned = ₹0

Test Case 5: Transaction Cancellation

Product = Notebook

Inserted Amount = ₹20

Result:

- Transaction Cancelled
- Refund Returned = ₹20

CHAPTER 7

ADVANTAGES

- Fast and reliable operation.
- Supports multiple payment methods.
- Automatic change calculation.
- Transaction cancellation feature.
- FPGA implementation capability.
- Scalable for additional products.

CHAPTER 8

APPLICATIONS

- Railway stations
- Educational institutions
- Shopping malls
- Airports
- Hospitals
- Smart retail stores

CHAPTER 9

FUTURE ENHANCEMENTS

- UPI payment integration.
- LCD display interface.
- Inventory management system.
- Multiple product quantities.
- GSM/Wi-Fi monitoring.
- Mobile application support.

CONCLUSION

The Digital Vending Machine was successfully designed and implemented using Verilog HDL. The system supports product selection, coin payment, online payment, automatic change return, and transaction cancellation. The FSM-based architecture ensures efficient and reliable operation. Simulation results verify that the vending machine functions correctly under different transaction scenarios. The design can be implemented on FPGA platforms for real-time deployment and serves as an excellent example of digital system design using Verilog HDL.